OPMIG-02: OpenPipeline Migration Guide: Part 2

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 2 of 10 | **Created:** December 2025 | **Last Updated:** 10/23/2026

Architecture & Key Concepts

Table of Contents

1. Data Flow Architecture
2. Processing Stages in Detail
3. Pipeline Types
4. Processor Types
5. Dynatrace Pattern Language (DPL)
6. Complete OpenPipeline Limits Reference
7. Entity Field Availability Timeline
8. Key Fields & Metadata
9. Exploring Your Pipeline Configuration
10. Understanding Processing Order
11. Summary: Key Architecture Concepts

Learning Objectives

By the end of this notebook, you will:

- ✅ Understand the complete OpenPipeline data flow architecture

- ✅ Learn detailed processing stage execution order
- ✅ Master DPL (Dynatrace Pattern Language) fundamentals
- ✅ **Know ALL OpenPipeline limits in comprehensive detail**
- ✅ Understand processor types and capabilities
- ✅ **Learn entity field availability timeline**
- ✅ Explore your pipeline configuration with DQL

Prerequisites

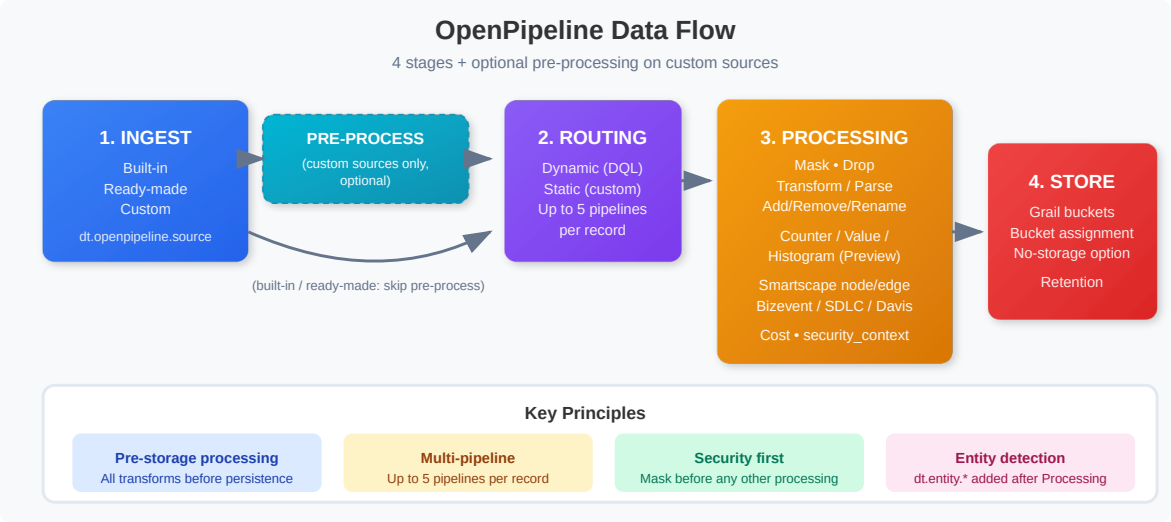
Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail — OpenPipeline runs in the SaaS environment; Managed is not covered by this series
API Access	`logs.read` token scope for DQL validation
Knowledge	OPMIG-01 or familiarity with Classic log ingestion concepts

---

Data Flow Architecture

Understanding how data flows through OpenPipeline is essential for designing effective pipelines.

Complete Data Flow



> **Stage model (verified 09/24/2026):** data flows Ingest → Routing → pipeline → Storage, with optional pre-processing on custom sources. Inside a pipeline, per *Processing in OpenPipeline*, *"The sequence of stages is fixed for all pipelines and cannot be modified."* Masking, dropping, parsing and transformation are processors in the first stage, **Processing**; **Bucket assignment** comes seventh, and **Metric extraction**, **Davis** and **Data extraction** run after it. The diagram's Processing box stands for that whole stage sequence.
>
> **Sources:** Processing in OpenPipeline (DT docs) — stage table, in execution order.

Key Principles

1. **Pre-storage Processing**: All transformations happen BEFORE data is written to Grail.
2. **Order Matters**: Processors execute in the order defined within each pipeline.
3. **One route, possibly several pipelines**: Each record takes the first matching route. Routed to a pipeline group's member pipeline, it is also processed by the group's base pipelines; data is extracted from one record in at most 5 pipelines.
4. **Entity Detection**: A documented set of entity fields — `dt.entity.service`, the Kubernetes and cloud-application entities, `dt.source_entity` and others — is added AFTER the Processing stage (full list under Field & Matching Restrictions).
5. **Immutable Storage**: Once stored, data cannot be modified (mask early!).

Processing Stages in Detail

Stage 1: Ingest

Records enter via an ingest source. The source is recorded as `dt.openpipeline.source`.

Source Type	Owner	Pre-processing	Routing
Built-in	OpenPipeline (view-only)	No	Dynamic only
Ready-made	Extensions	No	Dynamic only
Custom	User-defined	Yes (optional)	Dynamic or static

Stage 2: Pre-processing *(optional, custom sources only)*

A DQL transform applied to records from a custom source *before* routing. Use this to normalize raw data from third-party shippers into a common shape that all downstream pipelines can consume.

Stage 3: Routing

The routing stage determines which pipeline(s) process each incoming record.

Aspect	Description
Purpose	Match incoming data to appropriate pipelines
Timing	Before any pipeline processing
Configuration	Dynamic routing rules with matching conditions, or static assignment for custom sources
Order	First matching route wins — a record takes one route
Fallback	Unmatched data takes the default route (classic pipeline for logs and business events)

Matching Condition Examples:

```
k8s.namespace.name == "production"
log.source == "nginx"
matchesPhrase(content, "payment")
dt.openpipeline.source == "oneagent"
```

Matchers accept a subset of DQL — use `matchesPhrase` (whole tokens or phrases) or `matchesValue` with `*` wildcards (any substring, e.g. `matchesValue(content, "*/health*")`); `contains()` and `in()` are not enabled in matchers and the configuration will not save. Validate a matcher before saving it. (`contains()` remains valid *inside* a DQL processor definition — only matcher positions reject it.)

> **Sources:** DQL matcher in OpenPipeline (DT docs) — *"With Dynatrace powered by Grail, you can use Dynatrace Query Language (DQL) functions and logical operators in matchers."* The page documents `matchesPhrase`, `matchesValue`, `isNull` / `isNotNull` and `duration`; `contains()` rejected by the OpenPipeline matcher validator with *The function `contains()` isn't enabled.* (09/28/2026).

Stage 4: Pipeline — the Processing stage first

A pipeline is a fixed sequence of stages (full table in Understanding Processing Order). The first stage, **Processing**, holds the processors that edit the record:

Purpose	Processing-stage processors
Masking	DQL with `replacePattern` (or Remove fields)
Filtering	Drop record
Field & record manipulation	DQL (incl. `parse`), Add fields, Remove fields, Rename fields, Technology bundle
Enrichment	Inline lookup, GeoIP lookup (Early Access)

Metric extraction, Smartscape node/edge, event extraction, cost allocation, `dt.security_context` and bucket assignment are **not** part of the Processing stage — each is its own later stage in the fixed sequence.

> Recommended order *within* the Processing stage: mask → drop → parse → enrich.

> Everything else runs in a later, fixed stage.

Stage 5: Storage

The storage stage routes data to Grail buckets. Use the **No storage assignment** processor to skip retention entirely (useful when a record is only needed to extract a metric or event).

Aspect	Description
Bucket Routing	Direct data to specific buckets
Retention	Different retention periods per bucket
Cost Control	Route high-volume data to shorter retention or to no-storage

---

Pipeline Types

Processing in OpenPipeline defines three pipeline types, by who owns them:

Type	Owner	Access	What it is
Custom	User or user group	Owner-based; editable	The pipelines you build. All migrated processing lives here.
Ready-made	Extension	View-only	Created when an extension is installed, often with its own ingest source and route. To add processing, use a **pipeline group** with the ready-made pipeline as a member or base pipeline.
Built-in	OpenPipeline	View-only	Provided out of the box and *"generally cannot be modified within OpenPipeline."*

The default pipeline

The **default pipeline** is a built-in pipeline. It *"processes unassigned incoming data for storage"* — every record no route matches — and sends it to the configuration scope's

default bucket so it is *"not unintentionally dropped."* It is view-only: you cannot add processors to it. Processing you need on unmatched data belongs in a custom pipeline behind an explicit route.

Logs and business events are the exception. The default pipeline, in the docs' words, is *"not available for log and business event configuration scopes where the Classic pipeline is available."* There, records that fall through to the default route are processed by the **classic pipeline** and carry the pipeline id `logs:default` (or `bizevents:default`) in `dt.openpipeline.pipelines`. The query in the exploration section counts them; OPMIG-09 uses the same id to measure how much traffic has not been migrated yet.

Technology parsing for common formats is not a pipeline type. It is the **Technology bundle** processor, which you add to a custom pipeline (see Processor Types).

> 💡 **Best Practice:** Create focused pipelines for specific use cases rather than one large pipeline with complex conditionals.
>
> **Sources:** Processing in OpenPipeline (DT docs) — pipeline-type table; *"The default pipeline is a built-in pipeline that processes unassigned incoming data for storage."*, *"not available for log and business event configuration scopes where the Classic pipeline is available."*; Upgrade from classic pipeline to OpenPipeline (DT docs) — *"Data that doesn't match falls back to the default route and continues to be processed by the classic pipeline until you turn off the rules."*

Processor Types

DQL Processor

The DQL processor uses DQL commands to transform data. Available commands:

Command	Purpose	Example
`fieldsAdd`	Add new fields	`fieldsAdd environment = "prod"`
`fieldsRemove`	Remove fields	`fieldsRemove sensitive_field`
`fieldsRename`	Rename fields	`fieldsRename old_name = new_name`
`parse`	Extract with DPL patterns	`parse content, "LD:prefix INT:count"`

fieldsAdd Examples:

```
// Static value
| fieldsAdd environment = "production"

// Conditional value
| fieldsAdd severity = if(loglevel == "ERROR", "critical", else: "normal")

// Computed value
| fieldsAdd message_length = stringLength(content)

// From existing field
| fieldsAdd short_host = substring(host.name, from: 0, to: 10)
```

Drop Processor

Removes records matching specified conditions:

```
Matching Condition: loglevel == "DEBUG"
Result: All DEBUG logs are dropped before storage
```

Technology Bundle Processor

The **Technology bundle** processor *"matches records for the selected technology and processes them according to predefined context-sensitive processing statements."* Pick the technology (for example a web server, a runtime or a syslog source) and the bundle parses its records for you. The fields each bundle produces are defined by the bundle, not by you — open the processor's preview on a sample record to see exactly which fields it adds before you write anything downstream that depends on them.

> **Sources:** Processing in OpenPipeline (DT docs) — processor table, *Technology bundle*.

Dynatrace Pattern Language (DPL)

DPL is a powerful pattern matching language used in the `parse` command.

Core Matchers

Matcher	Description	Example Match
`INT`	Integer number	`42`, `-17`
`LONG`	Long integer	`1234567890123`
`DOUBLE`	Decimal number	`3.14`, `-0.5`
`IPADDR`	IPv4 or IPv6 address	`192.168.1.1`, `::1`
`IPV4ADDR`	IPv4 address only	`10.0.0.1`
`IPV6ADDR`	IPv6 address only	`2001:db8::1`
`TIMESTAMP`	Timestamp with format	`2024-01-15T10:30:00Z`
`LD`	Line data (to delimiter)	Any text until delimiter
`DATA`	Any data (greedy)	Consumes remaining text
`SPACE`	Whitespace	Spaces, tabs
`NSPACE`	Non-whitespace	Any non-space chars
`WORD`	Word characters	`hello`, `user123`
`JSON`	JSON structure	`{"key": "value"}`
`EOL`	End of line	Line terminator

Pattern Syntax

Element	Syntax	Description
Export to field	`MATCHER:fieldname`	Extract and name the field
Match only	`MATCHER`	Match but don't extract
Optional	`MATCHER?`	Matcher is optional
Literal	`'exact text'`	Match literal string
Alternatives	`('opt1'|'opt2')`	Match either option
Quantifier	`MATCHER{2,5}`	Match 2-5 times

> **Quantifier caveat:** `{n,m}` is not universal. `INT` accepts only `*` and `+` — `INT{3}` is rejected. For fixed-width digit runs use a character class: `[0-9]{3}`.

DPL Examples

```
// Extract IP and port from log
| parse content, "IPADDR:client_ip ':' INT:port"

// Extract user ID with flexible prefix, anywhere in the line
| parse content, "LD? ('user='|'userId='|'user_id=') NSPACE:user_id"

// Parse Apache-style log
| parse content, "IPADDR:client_ip SPACE '-' SPACE LD:user SPACE '['
LD:log_time ']'"

// Extract JSON payload
| parse content, "LD JSON:payload"

// Parse with optional port
| parse content, "IPADDR:ip (':' INT:port)?"

// Extract error code, anywhere in the line
| parse content, "LD? 'error_code=' INT:error_code"
```

> **`parse` matches from the start of the field.** A pattern that opens with a literal such as `'error_code='` returns **null** on every line that does not begin with it — no error, just an empty field. Prefix the pattern with `LD?` to skip any leading text (or none). And a trailing `LD:x` runs to the **end of the line**, not the end of the value: use `NSPACE:x` (stops at the next space) or a closing literal. Both verified with `data record(...)` on 10/02/2026; FAQ-15 covers DPL matching semantics in depth.

Timestamp Format Patterns

Symbol	Meaning	Example
`yyyy`	Year (4 digits)	2024
`MM`	Month (01-12)	01

Symbol	Meaning	Example
`dd`	Day (01-31)	15
`HH`	Hour 24h (00-23)	14
`mm`	Minute (00-59)	30
`ss`	Second (00-59)	45
`SSS`	Milliseconds	123

Complete OpenPipeline Limits Reference

This comprehensive reference contains ALL OpenPipeline limits you need to know for planning your migration.

Data Size & Volume Limits

Limit	Value	Behavior When Exceeded
Max record size (after processing)	16 MB	Record is **dropped**
Max request payload	10 MB per configuration scope	Request rejected (status code not stated on the limits page)
Working memory per record	Limited — no figure published †	Record dropped once processing memory is exhausted (reported as `buffer_overflow`)
Log attribute size	32 KB (4,096 characters per attribute in an event template)	Attribute value **truncated**
Max field name length	255 characters †	Field creation fails
Max string field length	32 KB †	Content truncated
Max array size	1000 elements †	Array truncated

Limit	Value	Behavior When Exceeded
Max nesting depth (JSON)	10 levels †	Deeper levels flattened

Processing Limits

Limit	Value	Impact
Max pipelines per record	5	Data can be extracted from one record in at most 5 pipelines (a pipeline group's base + member pipelines). After 5, data extraction stops but the record is still persisted.
Max processors per pipeline	1,000 (100 in a base pipeline)	Cannot add more processors to pipeline
Max DQL commands per processor	10 commands †	Split complex logic into multiple processors
Max parse operations per processor	100 patterns †	Create additional parse processors
Max fields per record	500 fields †	Additional fields ignored
Processing timeout per record	30 seconds †	Record dropped if exceeded
Max processor name length	100 characters †	Validation error

Timestamp Constraints

Data Type	Earliest accepted timestamp (older → dropped before processing)	Timestamp more than 10 min in the future
Logs	Ingest time minus 24 hours. From **SaaS 1.348** (rollout start 10/12/2026, staged tenant rollout): 72 hours — verify the new window has reached your tenant; 24 hours remains the working value until then	**Adjusted** to ingest time + 10 min
Spans	60 minutes past (end time) †	Not adjusted (the adjustment doesn't apply to spans)
Events	Ingest time minus 24 hours	**Adjusted** to ingest time + 10 min
Business Events	Ingest time minus 24 hours	**Adjusted** to ingest time + 10 min
Metrics	Ingest time minus 1 hour	**Adjusted** to ingest time + 10 min

> ⚠️ **Critical:** Historical data imports require workarounds. Contact Dynatrace support for backfilling options.

Pipeline Group Limits *(per group)*

Limit	Value	Notes
Pipeline slots per group	10	Per `/reference/limits`
Member pipelines per group	1,000	Per `/reference/limits`

Pipeline & Routing Limits

Limit	Value	Scope
Max custom pipelines	100 pipelines	Per configuration scope (logs, spans, etc.)

Limit	Value	Scope
Max pipeline groups	100	Per configuration scope
Max dynamic routes	100 routes	Per configuration scope
Max ingest sources	100	Per configuration scope
Max conditions per route	10 conditions †	Combine with AND/OR operators
Max pipeline name length	100 characters †	Validation error
Max route name length	100 characters †	Validation error

Extraction Limits

Limit	Value	Notes
Max metric extractions per pipeline	10 †	Value + counter metrics combined
Max event extractions per pipeline	5 †	All event types combined
Max bizevent extractions per pipeline	3 †	Business events only
Max dimensions per metric	10 dimensions †	Keep cardinality low
Metric key length	250 characters †	Validation error
Event type name length	100 characters †	Validation error

DQL & DPL Limits

Limit	Value	Context
DQL processor script length	8,192 characters	Per `/reference/limits` (May 2026 doc)
Processor matching condition length	4,096 characters (Settings API); 1,500 for legacy Configurations API / Classic pipelines	Per `/reference/limits` (Jun 17, 2026 doc)

Limit	Value	Context
Max DPL pattern length	4 KB †	Per parse pattern
Max captured groups per parse	100 groups †	Use multiple parse operations
Max alternatives in pattern	50 alternatives †	`(opt1|opt2|...|opt50)`

Bucket & Storage Limits

Limit	Value	Notes
Max custom buckets	80 by default; 250 from SaaS 1.346 (staged tenant rollout from 08/25/2026)	Per environment. The upgrade guide still states 80, the Grail *organize data* page and the 1.346 release note state 250 — check which default your tenant has before planning to either number
Min retention period	1 day	Per custom bucket
Max retention period	10 years, with an additional week	Per custom bucket
Bucket name length	100 characters †	Alphanumeric + underscore

> **Sources:** Data partitioning — upgrade best practices (DT docs) — *"The default bucket limit per environment is 80 buckets, which is typically sufficient for up to 5 TB/day per table."*; Organize data (DT docs) — *"The default limit per environment is 250 custom buckets."*; SaaS 1.346 (DT docs) — *"Dynatrace now supports up to 250 custom Grail buckets by default"*. Retention: Organize data (DT docs) — *"For custom buckets, the possible retention periods range from 1 day to 10 years, with an additional week."*

Rate Limits

Endpoint	Limit	Per
Log Ingest API	500 requests/min †	Per token
OTLP Endpoint	1000 requests/min †	Per token
Config API	100 requests/min †	Per token

Field & Matching Restrictions

View-only fields — *"editing via OpenPipeline is not supported"*:

```
dt.ingest.*          - Ingestion metadata
dt.openpipeline.*    - Pipeline processing metadata
dt.retain.*          - Retention information
dt.system.*          - System metadata (bucket, etc.)
```

Fields added after the Processing stage — usable *"only in stages after the Processing stage, but not in pre-processing, routing, or the Processing stage"*:

```
dt.entity.service
dt.entity.kubernetes_cluster, dt.entity.kubernetes_node,
dt.entity.kubernetes_service
dt.entity.cloud_application, dt.entity.cloud_application_instance (and the
cloud-application namespace field)
dt.entity.custom_device, dt.entity.aws_lambda_function,
dt.entity.&lt;genericEntityType&gt;
dt.source_entity
dt.kubernetes.cluster.id, dt.kubernetes.cluster.name, k8s.cluster.name
dt.env_vars.dt_tags
dt.process.name  (classic pipelines only — use dt.process_group.detected_name
before Processing)
```

`k8s.cluster.name` is available before Processing when the OneAgent Log module runs in standalone mode (OneAgent 1.309, Dynatrace Operator 1.4.2+). `dt.entity.host`, `dt.entity.process_group` and `dt.entity.process_group_instance` are **not** on this list — the limits page does not restrict them, so do not design around them being missing; preview a processor on a real record to confirm what is present at that point.

Metric scope is stricter. In the metric configuration scope the limits page lists `dt.entity.*` — every entity field, host included — under *"Fields excluded from dynamic*

route matching conditions and in the Processing stage". The list above applies in every configuration scope; in the metric scope, additionally keep all `dt.entity.*` out of routing conditions and Processing-stage processors.

> 💡 **Design Pattern:** For the fields on the list above, you cannot:
> - Use them in routing conditions
> - Use them in matching conditions during processing
> - Modify them with fieldsAdd/fieldsRename
>
> You CAN use them in the stages after Processing — for example Bucket assignment and the extraction stages (metrics, events).

The limits page does not publish a general ban on creating `dt.*` fields, but that namespace belongs to the Dynatrace semantic dictionary. In community practice, custom fields use your own prefix (or none) so they never collide with a field Dynatrace adds later.

> **Sources:** OpenPipeline limits (DT docs) — *Restricted fields*: *"The following fields can be viewed-only; editing via OpenPipeline is not supported."*; *"The following fields are added after the Processing stage when Dynatrace runs its entity detection."* (list read 10/02/2026); *Metric configuration scope*: *"Fields excluded from dynamic route matching conditions and in the Processing stage"* — `dt.entity.*` (read 10/14/2026).

Workarounds for Common Limit Issues

Problem	Workaround
>1,000 processors needed (>100 in a base pipeline)	Split the processing across pipelines — for example member pipelines in a pipeline group
>10 DQL commands	Break into multiple processors (order matters!)
>100 parse patterns	Use multiple parse processors in sequence
>16MB after processing	Drop unnecessary fields, reduce field sizes
>5 pipelines in a group needed	Consolidate processing logic, use conditional processors
>10 metric dimensions	Reduce cardinality, use separate metrics

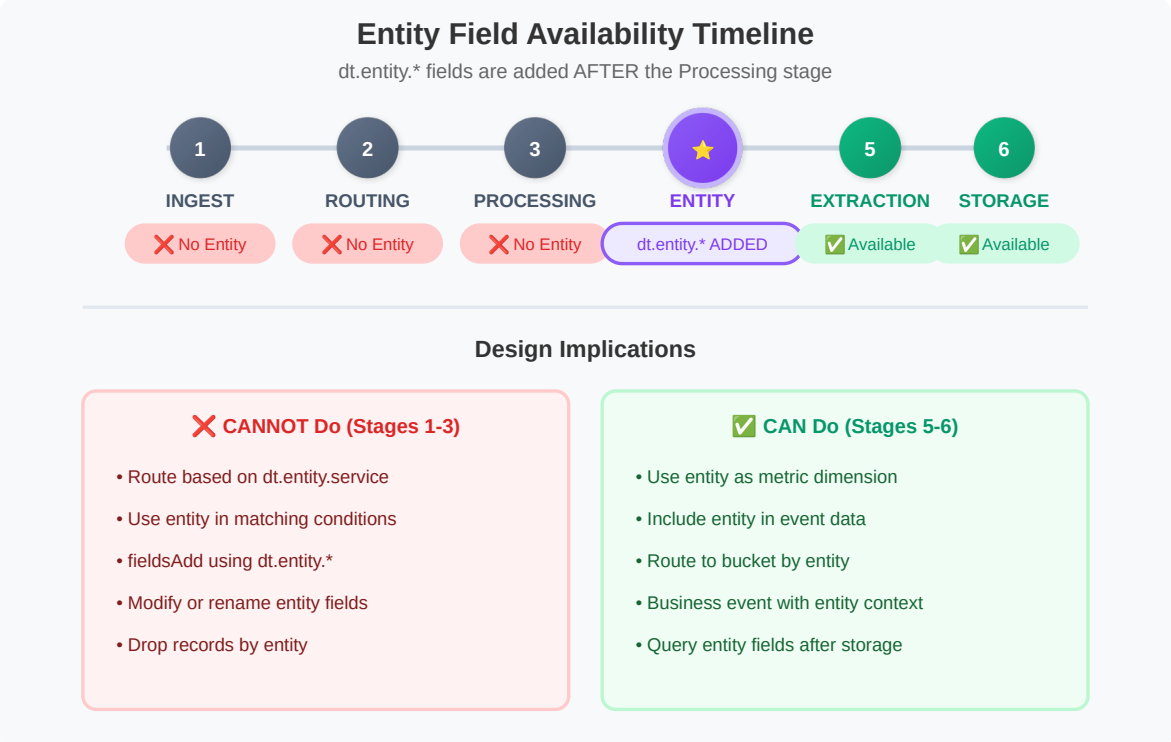
† Not stated on the *OpenPipeline limits* page (read 09/28/2026) — treat as community-reported and verify in your tenant before planning to it. Every unmarked number in the tables above is on that page.

> **Sources:**
>
> - OpenPipeline limits (DT docs) — *"If the timestamp is more than 10 minutes in the future, it's adjusted to the ingest server time plus 10 minutes."*; *"The request payload size maximum limit is 10 MB per configuration scope."*; *"Once the available processing memory is exhausted, the record is dropped."*
> - What's new in SaaS 1.348 (DT docs) — *"The log ingestion pipeline now accepts log records with timestamps up to 72 hours in the past, extended from the previous 24-hour limit."*

Entity Field Availability Timeline

Understanding WHEN entity fields become available is critical for pipeline design.

Processing Stage Timeline



Practical Examples

❌ WRONG - Cannot route based on entity

```
Dynamic Route:
  Condition: dt.entity.service == "SERVICE-123"  ❌ FAILS

Why: Entity fields not available during routing
```

❌ WRONG - Cannot use entity in processing

```
DQL Processor:
  fieldsAdd service_name = dt.entity.service  ❌ FAILS

Why: Entity fields not available during processing
```

✅ CORRECT - Use entity in metric extraction

```
Metric Extraction:
  Key: log.request.duration
  Value: duration_ms
  Dimensions: dt.entity.service, dt.entity.host  ✅ WORKS

Why: Entity fields available in extraction stage
```

✅ CORRECT - Use entity for bucket routing

```
Bucket Routing (in Storage stage):
  Condition: dt.entity.service == "CRITICAL-SERVICE"
  Bucket: critical_logs  ✅ WORKS

Why: Entity fields available in storage stage
```

Design Patterns

Pattern 1: Route by source, extract metrics by entity

```
Routing: log.source == "payment-service"  (uses source field)
Processing: Parse payment data
Extraction: Create metric with dt.entity.service dimension  ✅
```

Pattern 2: Enrich with custom field, then use entity

```
Processing: fieldsAdd app_tier = "frontend"  (custom field)
Extraction: Dimensions: app_tier, dt.entity.service  ✅
```

Pattern 3: Cannot mix entity with routing

```
❌ Route based on entity → Use workaround:
   - Route based on source/content patterns instead
   - Use service name string (not entity ID) if available
```

Key Fields & Metadata

OpenPipeline-Specific Fields

Field	Description	Example Value
`dt.openpipeline.source`	Data source identifier — for built-in API sources, the endpoint path	`oneagent`, `/api/v2/logs/ingest`, `/api/v2/otlp/v1/logs`
`dt.openpipeline.pipelines`	Pipeline(s) that processed record — an **array** of `"<scope>:<pipeline id>"` strings	`["logs:<pipeline id>"]`
`dt.system.bucket`	Grail storage bucket	`default_logs`, `custom_logs`

Run the first query in Exploring Your Pipeline Configuration to see the source values in your own tenant. Because `dt.openpipeline.pipelines` is an array,

`dt.openpipeline.pipelines == "name"` is always false — filter with `matchesValue(dt.openpipeline.pipelines, "*name*")` instead.

> **Sources:** Data flow (DT docs) — ingest sources *"are defined by a name and a path (dt.openpipeline.source)"*.

Log Fields

Field	Description
`timestamp`	Log timestamp (primary time field)
`content`	Log message content
`loglevel`	Log level (ERROR, WARN, INFO, DEBUG)
`status`	Status string (alternative to loglevel)
`log.source`	Log source identifier
`log.iostream`	Stream type (stdout, stderr)

Entity Context Fields

Field	Description
`dt.entity.host`	Host entity ID
`dt.entity.process_group`	Process group entity ID
`dt.entity.process_group_instance`	Process instance entity ID
`dt.entity.service`	Service entity ID
`host.name`	Host name (string)
`process.executable.name`	Process executable name

> ⚠️ **Important:** `dt.entity.service`, the Kubernetes and cloud-application entity fields and `dt.source_entity` are added AFTER the Processing stage, so you

> cannot use them in routing or processing conditions. The full list is under Field & Matching Restrictions.

Exploring Your Pipeline Configuration

Use these queries to understand how your environment is configured.

```
// View data sources currently sending to OpenPipeline
// Shows the distribution of data by ingestion source
fetch logs, from: now() - 24h
| summarize {record_count = count()}, by: {dt.openpipeline.source}
| sort record_count desc
```

```
// Analyze which pipelines are processing your logs
// Helps verify routing is working correctly
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {record_count = count()}, by: {dt.openpipeline.pipelines}
| sort record_count desc
```

```
// Check bucket distribution for stored logs
// Verify data is routing to expected buckets
fetch logs, from: now() - 24h
| summarize {record_count = count()}, by: {dt.system.bucket}
| sort record_count desc
```

```
// Analyze pipeline processing by source and pipeline
// Shows the relationship between sources and pipelines
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| summarize {record_count = count()}, by: {dt.openpipeline.source,
dt.openpipeline.pipelines}
| sort record_count desc
| limit 25
```

```
// Check for span processing through OpenPipeline
// Spans also flow through OpenPipeline
fetch spans, from: now() - 24h
| summarize {span_count = count()}, by: {dt.openpipeline.source}
| sort span_count desc
```

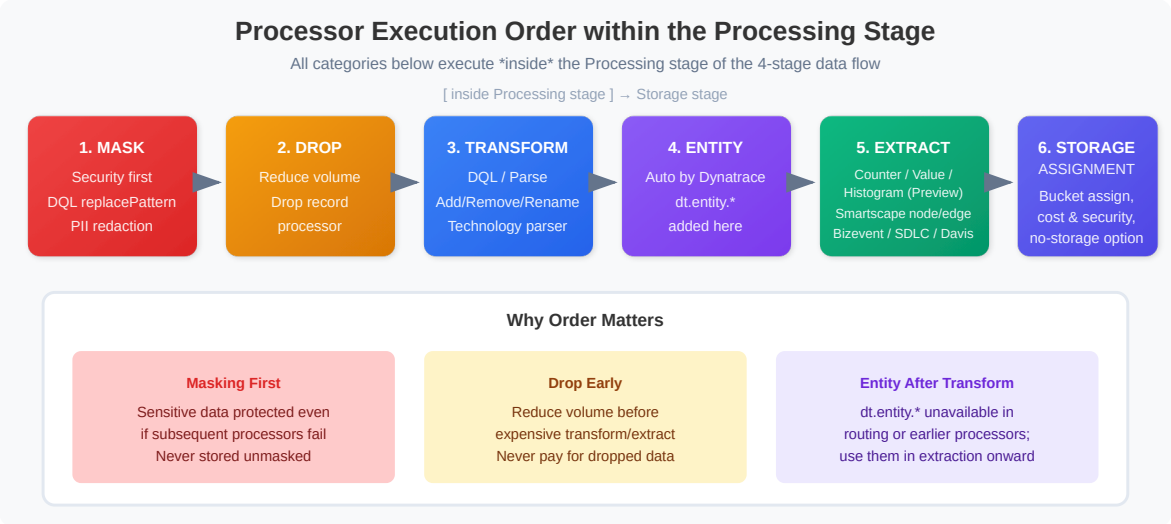
```
// View pipeline processing over time
// Helps identify volume patterns and processing trends
fetch logs, from: now() - 24h
| filter isNotNull(dt.openpipeline.pipelines)
| makeTimeseries {record_count = count()}, by: {dt.openpipeline.pipelines},
interval: 1h
```

```
// Records that fell through to the default route, last 24 hours.
// During a migration these are processed by the classic pipeline (id
"logs:default").
// dt.openpipeline.pipelines is an array, so compare with in(), never ==.
fetch logs, from: now() - 24h
| filter in(dt.openpipeline.pipelines, "logs:default")
| summarize {unrouted_count = count()}, by: {log.source}
| sort unrouted_count desc
| limit 20
```

Understanding Processing Order

Two orders are in play, and only one of them is yours to choose:

1. **The stage sequence is fixed.** Per *Processing in OpenPipeline*, *"The sequence of stages is fixed for all pipelines and cannot be modified."* Every pipeline runs **Processing** → Smartscape node → Smartscape edge → Permission → Product allocation → Cost allocation → **Bucket assignment** → **Metric extraction** → **Davis** → **Data extraction**. Masking, dropping, parsing and transformation are all processors in the first stage; there is no separate masking or filtering stage, and the extraction stages run *after* bucket assignment.
2. **Processor order within a stage is the order you configure** — *"each processor output becomes the input for the next one."*



> 💡 **Tip:** Masking is not automatically first. Inside the Processing stage, place masking processors **before** any processor that copies, parses or extracts from the sensitive field — a processor listed ahead of the masking processor sees the raw value. Records reach Bucket assignment and storage only after the whole Processing stage has run. The diagram groups processors by purpose; the execution order the platform enforces is the stage sequence above.
>
> **Sources:** Processing in OpenPipeline (DT docs) — stage table, in execution order; *"The processor order in the stage; each processor output becomes the input for the next one."*

Summary: Key Architecture Concepts

Concept	Key Points
Data Flow	Ingest → (Pre-process) → Route → pipeline (fixed stage sequence) → Store
Pre-storage	All processing happens before data is persisted
Source Types	Built-in / Ready-made / Custom — only custom sources support pre-processing and static routing
Routing	Dynamic (DQL matcher) or static (custom sources only)

Concept	Key Points
Processing Order	Fixed stages: Processing (mask, drop, parse, transform — in the order you list them) → Smartscape node/edge → Permission → Product and cost allocation → Bucket assignment → Metric extraction → Davis → Data extraction
Entity Detection	Happens AFTER processing, BEFORE most extraction processors run
Multi-pipeline	One route per record; pipeline groups add base pipelines; extraction in at most 5 pipelines (`/reference/limits`)
DPL	Powerful pattern language for parsing
Buckets	Control retention and cost at the storage assignment step; or skip with No storage assignment

---

Next Steps

Now that you understand OpenPipeline architecture, continue with:

Notebook	Focus Area
OPMIG-03	Migration Assessment & Planning
OPMIG-04	Pipeline Configuration Fundamentals
OPMIG-05	Routing & Bucket Management
OPMIG-06	Processing, Parsing & Transformation

References

- OpenPipeline Data Flow
- OpenPipeline Processing
- Dynatrace Pattern Language
- OpenPipeline Limits

- DPL Architect Tool

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.